

LATENT PLANNING VIA EMBEDDING ARITHMETIC: A CONTRASTIVE APPROACH TO STRATEGIC REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Planning in high-dimensional decision spaces is increasingly reframed as a problem of learned representation rather than one of explicit search. We show that, in a perfect-information setting equipped with dense evaluation labels, planning need not rely on an explicit policy, a value head, or a learned dynamics model, but can be conducted directly within an evaluation-aligned embedding space. We introduce SOLIS, which constructs such a space through supervised contrastive learning, so that proximity between embeddings encodes similarity of outcome and a single global *advantage vector*—the displacement from the mean losing embedding to the mean winning embedding—orients the geometry from losing toward winning regions. Action selection then reduces to ranking candidate continuations by their alignment with this direction, recasting the inner loop of planning as vector arithmetic in latent space. Instantiated as a chess engine and trained on five million Stockfish-labeled ChessBench positions, SOLIS attains a rating exceeding 2500 Elo against time-limited Stockfish with only a shallow embedding-guided search, and in depth-matched play it matches or surpasses a strong Stockfish baseline while expanding far fewer nodes per move—one to two orders of magnitude below Stockfish’s worst-case expansion at the same depth. Within the regime we study, these results indicate that a substantial portion of the strength conventionally attributed to broad tree expansion can instead be recovered from the quality of the learned representation, and they position evaluation-aligned latent planning as a lightweight, interpretable complement to rollout- and dynamics-model-based search whose extension beyond chess we identify as a direction for future work.

1 INTRODUCTION

Planning in high-dimensional state spaces remains a central challenge in artificial intelligence. Conventional reinforcement-learning methods rely on either explicit forward search or learned policies, yet both scale poorly as the combinatorial breadth of the action space grows, because the number of candidate action sequences expands exponentially with the planning horizon (Jiang et al., 2023; Gieslmann and Pokorny, 2022). A prominent line of work confronts this intractability through *latent planning*, which reasons over compact learned representations rather than raw states and thereby contains search complexity while enabling decision-making at a more abstract, human-like level (Hafner et al., 2019; 2020; Schrittwieser et al., 2020). Existing latent planners, however, continue to couple the learned representation with auxiliary machinery—a learned dynamics model together with a policy or value head—and it is this auxiliary machinery, rather than the representation itself, that performs the decisive computation. We ask whether the representation can instead be made to carry the planning signal on its own.

We answer this question in the affirmative: a learned embedding space can itself support planning once decision-making is cast as directed interpolation between regions of differing advantage. To this end we introduce **SOLIS** (*Strategic Optimization via Latent Interpolative Spaces*), a chess engine built on a transformer encoder trained with supervised contrastive learning to embed positions into an evaluation-aligned latent space. Within this space, distance encodes similarity of outcome and displacement along a distinguished axis corresponds to progress toward more favorable states. SOLIS exploits this geometry by ranking candidate moves according to their alignment with a single global *advantage vector* that orients the latent space from losing toward winning regions, thereby dispensing entirely with the learned policies, value heads, and dynamics models on which prior la-

tent planners depend. The evaluative signal is thus relocated from an auxiliary decision head into the representation, and action selection becomes an inner product rather than a rollout.

Despite its minimalism, this formulation proves strong even at modest search depths. Under 50 ms-per-move blitz conditions a five-ply embedding-guided search suffices for SOLIS to exceed 2500 Elo, and in depth-matched play at search depths $S \in \{3, 4, 5\}$ with a fixed branching factor it matches or surpasses a strong Stockfish baseline while expanding substantially fewer nodes per move—even though the baseline is granted its default unrestricted search width. Two further empirical regularities sharpen this result: search width confers little benefit beyond three candidate continuations, which localizes SOLIS’s strength in the fidelity of the learned geometry rather than in the breadth of expansion, and a compact 0.8M-parameter encoder matches a model roughly fifty times its size once sufficient depth is available. The latent trajectories of complete games corroborate these quantitative findings and render the planning process legible: decisive games trace smooth, near-monotone paths through the embedding space toward the winning region, whereas balanced games remain near the neutral center.

Whereas latent planners couple a learned dynamics model with policy or value heads (Hafner et al., 2019; 2020; Schrittwieser et al., 2020), SOLIS relies solely on the geometry of an evaluation-aligned representation for action selection; this reallocation of the evaluative signal from an auxiliary head into the representation constitutes our central methodological departure from prior work. We make three contributions.

- **Planning as embedding arithmetic.** We formulate action selection as alignment with a single global advantage direction, reducing the inner loop of planning to a vector projection and eliminating the auxiliary policy, value, and dynamics components that latent planners typically require.
- **An evaluation-aligned contrastive geometry with two scoring rules.** We show that a supervised contrastive objective, trained solely on Stockfish win-probability labels, is sufficient to induce a latent geometry in which this arithmetic is well posed, and we introduce two complementary rules for converting the geometry into move scores: an unanchored global-directional rule and an anchored relative-projection rule.
- **An empirical characterization of representation-driven search in chess.** Through more than 600 rated games per configuration and depth-matched matches against Stockfish, we demonstrate that this search is competitive at a small fraction of the expansion budget, that its strength scales predictably with depth while saturating in width, and that its latent trajectories admit a direct geometric interpretation.

We develop and evaluate the method entirely within chess, a perfect-information domain in which legal moves, state transitions, and evaluation labels are all available; whether the same vector-based latent planning transfers to domains with sparser or less reliable evaluations is a question we leave open and return to in Section 5.

2 BACKGROUND

Planning in very large action spaces is formidable precisely because the number of candidate action sequences grows exponentially with each additional decision step, so that exhaustive lookahead rapidly becomes intractable. This section establishes the conceptual background against which SOLIS is situated: we review the Markov decision process formalism, the distinction between planning and reinforcement learning that motivates our design, the trajectory of chess-engine research from handcrafted search to learned evaluation, and the contrastive-learning machinery on which our representation rests.

2.1 MARKOV DECISION PROCESSES

Sequential decision-making tasks are formalized as Markov Decision Processes (MDPs), specified by the tuple $\{\mathcal{S}, \mathcal{A}, T, R, p(s_0), \gamma\}$, in which $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, T the transition model, R the reward function, $p(s_0)$ the initial-state distribution, and $\gamma \in [0, 1]$ the discount factor governing the weighting of future rewards (Puterman, 2014). A policy $\pi : \mathcal{S} \rightarrow p(\mathcal{A})$ maps each state to a probability distribution over actions.

In this setting, a *model* denotes reversible access to the MDP dynamics: given a state–action pair, it returns the resulting next state and reward (Moerland et al., 2022). For perfect-information games such as chess and Go the rules furnish an exact model directly, whereas in domains with unknown dynamics the model must be approximated from data (Gieselsmann and Pokorný, 2022). This distinction matters for the present work, since SOLIS is developed in the former regime, where transitions and evaluation labels are externally available.

Planning and reinforcement learning are distinguished chiefly by how they exploit this access to the dynamics (Moerland et al., 2022). Planning methods presuppose a model and simulate forward from the current state to select an action, in analogy to human deliberation over anticipated consequences (Russell and Norvig, 1995); because the resulting solution is defined relative to the current state, it is *local* and is discarded once the action is executed. Reinforcement-learning methods, by contrast, typically lack reversible access to the dynamics (Sutton and Barto, 1998), and, interacting with the environment one step at a time, they instead learn a *global solution*—a policy or value function—that generalizes across the entire state space.

Model-based reinforcement learning unifies these two ingredients, employing a model while simultaneously learning a global solution. Following Moerland et al. (2022), it admits two forms: model-based RL with a *learned* model (e.g., Dyna (Sutton, 1991)), in which both the dynamics and the global solution are estimated from data, and model-based RL with a *known* model (e.g., AlphaZero (Silver et al., 2017)), in which the dynamics are supplied and only the global solution is learned. SOLIS shares the known-model setting of the latter but, as we detail below, departs from both by declining to learn a global policy or value function at all.

2.1.1 MODEL-BASED RL WITH A LEARNED MODEL

A substantial body of work occupies the category of model-based RL with a learned model, in which the dynamics and the global solution are estimated jointly. PlaNet (Hafner et al., 2019) learns a latent dynamics model and pairs it with a learned global solution; Dreamer (Hafner et al., 2020) and its successors DreamerV2 (Hafner et al., 2022) and DreamerV3 (Hafner et al., 2024) scale this recipe to progressively more complex tasks while retaining explicit global value functions and policies. MuZero (Schrittwieser et al., 2020) likewise learns a latent dynamics model but couples it with tree search to strengthen planning, again training a global policy and value function, and Dyna (Sutton, 1991) and MBPO (Janner et al., 2021) similarly leverage learned models to support value or policy learning. A common thread unites these methods: the learned model is instrumental to fitting an auxiliary policy or value estimator rather than serving as the locus of decision-making itself—the very dependence that SOLIS removes.

2.1.2 MODEL-BASED RL WITH A KNOWN MODEL

When the environment dynamics are given, the algorithm can concentrate on coupling search with learning. AlphaZero (Silver et al., 2017) adopts the exact game rules as its model and combines Monte Carlo tree search (MCTS) with learned policy and value heads; SAVE (Hamrick et al., 2020) likewise assumes a simulator and integrates Q-learning with MCTS, using a learned Q-function as a search prior while amortizing the search-produced Q-estimates back into the network. In both cases search remains indispensable and the learned components exist to guide or refine it. SOLIS instead pushes the evaluative signal entirely into the representation, so that a shallow search over a learned geometry stands in for the deep expansion these methods require.

2.1.3 LATENT PLANNING

Latent planning simplifies decision-making by operating in a compressed representation rather than the raw state space. PlaNet (Hafner et al., 2019), Dreamer (Hafner et al., 2020), and MuZero (Schrittwieser et al., 2020) pair a latent dynamics model with value or policy heads, enabling planning through simulated rollouts in the learned space; Expansive Latent Space Trees (ELAST) (Gieselsmann and Pokorný, 2022) cast latent planning as explorative tree search, combining contrastive representations with sampling-based expansion for long-horizon control; and the Trajectory Autoencoding Planner (TAP) (Jiang et al., 2023) learns a compact latent action space and plans over discrete action codes. Across these methods the learned representation supports, but does not by

itself constitute, the planning computation, which still rests on simulated dynamics and an auxiliary decision head.

SOLIS departs from this pattern at its foundation: it trains neither a policy nor a value head, learns no dynamics model, and instead distills the evaluation signal directly into the geometry of the representation. Planning then reduces to traversing that geometry toward regions aligned with favorable outcomes, using the encoder alone at inference. Our experiments establish that, in the chess setting we study, this evaluation-aligned embedding is by itself sufficient to drive competitive planning, without any of the auxiliary components on which the latent planners above depend.

2.2 CHESS ENGINES

Computer chess reached a landmark in 1997, when IBM’s Deep Blue defeated the reigning world champion, Garry Kasparov, in a head-to-head match. Early versions of Stockfish (Romstad et al., 2008) inherited Deep Blue’s paradigm—handcrafted evaluation functions paired with deep alpha-beta tree search—a methodology powerful enough to achieve superhuman play, yet whose ceiling was ultimately set by the collective chess expertise of the humans encoding the static evaluation.

The rating bottleneck imposed by handcrafted evaluation motivated a line of research that dispenses with human guidance. AlphaZero (Silver et al., 2017) and its open-source counterpart Leela Chess Zero (Pascutto and Linscott, 2018) advanced to engines that learn to evaluate positions entirely through self-play reinforcement learning, using no prior knowledge beyond the rules of chess; in parallel, modern versions of Stockfish incorporated an efficiently updatable neural-network evaluation module (NNUE) (Nasu, 2018) that accelerates position evaluation while preserving traditional alpha-beta search.

Although these advances have produced engines that far exceed human strength, the underlying search procedure has remained essentially fixed. It still follows the principles first articulated by v. Neumann (1928) and Shannon (1950) and later realized in Deep Blue, relying on deep alpha-beta expansion through millions of states (Lai, 2015). We take a different route, asking whether a contrastively learned representation can subsume much of this search: rather than expanding the game tree exhaustively, SOLIS recasts planning as directed traversal of a high-dimensional embedding space. Because SOLIS is trained from Stockfish evaluations and benchmarked against Stockfish, we are careful throughout to separate what this comparison establishes—that a learned geometry can substitute for broad expansion at matched depth—from what it does not, namely independence from the evaluator whose labels supervise the representation.

2.3 CONTRASTIVE LEARNING

Contrastive representation learning trains an encoder to place semantically similar inputs close together, and dissimilar inputs far apart, in a latent space (Chen et al., 2020). Formally, given a batch I of training examples containing an anchor input x_i , an encoder $f(\cdot)$ maps the anchor to a normalized embedding $z_i = f(x_i)$. A positive pair (z_i, z_j) consists of two semantically similar inputs, whereas every remaining pair in the batch is treated as a negative; the objective thereby shapes the geometry of the embedding space by contrasting each anchor against its positives and negatives simultaneously. The choice of what counts as “similar” is therefore the lever through which supervision enters the geometry—a lever we exploit by defining similarity directly on Stockfish win probability.

A common contrastive objective is InfoNCE (van den Oord et al., 2019), defined as:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \in A(i)} \exp(\text{sim}(z_i, z_k)/\tau)}, \quad (1)$$

where $A(i) = I \setminus \{i\}$ is the set of all batch samples excluding the anchor i , $\text{sim}(\cdot, \cdot)$ denotes a similarity metric, and τ is a scalar temperature controlling the sharpness of the induced distribution. In the self-supervised regime the positive z_j is an augmented view of the anchor z_i , so that no labels are required (Chen et al., 2020; Khosla et al., 2021).

When labels are available, the supervised contrastive loss (SupCon) (Khosla et al., 2021) generalizes InfoNCE by admitting multiple positives per anchor, and is defined as:

$$\mathcal{L}_{\text{sup}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \in A(i)} \exp(\text{sim}(z_i, z_a)/\tau)}, \quad (2)$$

where $P(i)$ denotes the set of positives associated with anchor i and $A(i)$ is defined as above. Because enlarging the number of positive and negative pairs has been shown to improve the quality of the learned representation (Khosla et al., 2021; Chowdhury et al., 2024), the multi-positive formulation is well suited to our setting: we exploit it directly by constructing, for each anchor, an evaluation-margin set of positives whose win probabilities lie within a fixed tolerance of the anchor’s.

3 METHODS

We now specify the four components of the approach in the order in which they are applied: the tokenization of chess positions, the transformer encoder architecture, the supervised contrastive training procedure that induces the evaluation-aligned latent space, and the embedding-guided action selection performed at inference. The first three components together determine the geometry of the embedding space, and the fourth converts that geometry into moves; the design principle throughout is to concentrate the evaluative signal in the representation so that inference-time computation remains minimal.

3.1 INPUT REPRESENTATION AND TOKENIZATION

Each chess position is represented by its Forsyth–Edwards Notation (FEN) ASCII string, which compactly encodes piece placement, side to move, castling rights, the en passant target, the half-move clock, and the full-move counter. Following the tokenization scheme of Ruoss et al. (2024), we expand the run-length-encoded FEN into a fixed-length sequence of 77 tokens, yielding a uniform input length across positions. Encoding a single position in isolation, as Ruoss et al. (2024) observe, renders the task non-Markovian: move history is discarded, even though it is decisive in the special cases of threefold repetition and the fifty-move rule. This is a deliberate simplification that keeps the input representation uniform, and we quantify its cost—a specific and well-understood failure mode—in Section 5, where we also indicate the rule-aware history features that would remove it.

3.2 ENCODER ARCHITECTURE

Our encoder is a multi-layer transformer (Vaswani et al., 2023) adapted to tokenized chess positions. We instantiate two variants, SOLIS Base and SOLIS Mini, which span roughly two orders of magnitude in parameter count and thereby let us isolate how representation capacity, at fixed depth, translates into planning strength; their configurations are summarized in Table 1.

Table 1: Transformer configurations for the two SOLIS variants evaluated in our experiments. SOLIS Mini (0.8M parameters) and SOLIS Base (41M parameters) share a common six-layer depth but differ by roughly two orders of magnitude in parameter count, spanning embedding dimension D , attention-head count, and MLP width. Holding depth fixed while varying width isolates the effect of representation capacity on planning strength.

Model	Layers	Embedding dim. D	Heads	MLP Size	Params
SOLIS Mini	6	128	8	256	0.8M
SOLIS Base	6	1024	16	1024	41M

A learned token-embedding matrix maps each token to a D -dimensional vector. A dedicated classification token (CLS) (Devlin et al., 2019; Dosovitskiy et al., 2021) is prepended to the sequence to aggregate global information across the position, and, since the sequences are of fixed length, a learned positional encoding is added to every token embedding (Ruoss et al., 2024). The augmented sequence is then processed by a stack of transformer encoder layers with GELU (Hendrycks and

Gimpel, 2023) activations and dropout (Srivastava et al., 2014). Finally, the hidden state at the CLS position is extracted, passed through a linear projection, and ℓ_2 -normalized to yield the position embedding z , the sole object on which all subsequent scoring operates.

3.3 SUPERVISED CONTRASTIVE TRAINING

We train the encoder on 5 million positions sampled uniformly at random from the ChessBench dataset (Ruoss et al., 2024), each annotated with a Stockfish-evaluated win probability for the player to move. To obtain a consistent evaluative axis we normalize every annotation to White’s perspective, so that values near 1.0 denote a decisive advantage for White and values near 0.0 a decisive advantage for Black. This single scalar target is the only supervision the model receives; it is precisely this label that the contrastive objective distills into the geometry of the embedding space, and its origin in Stockfish’s static evaluation is a property we revisit when interpreting the results.

Positive pairs are defined through an evaluation margin δ : two positions are deemed similar when their win probabilities differ by less than δ . For each anchor we precompute the set of positions satisfying this criterion and sample five of them as positives during training, and at the batch level we construct the positive mask by marking every pair whose evaluation difference falls below δ . This margin-based construction attracts positions of nearly equal evaluation while repelling those separated by a larger evaluative gap. Because similarity is defined directly on the win probability, the resulting geometry inherits a total ordering from losing to winning positions—exactly the structure that renders directional planning along a single advantage vector well posed. The construction thus converts a scalar regression target into a relational one, trading the pointwise fit of a value head for a globally consistent ordering of positions.

We optimize the supervised contrastive (SupCon) objective (Khosla et al., 2021) using these batch masks to designate positive and negative pairs. Pairwise affinities are computed as cosine similarities between ℓ_2 -normalized embeddings, with a temperature $\tau = 0.07$ (Girdhar et al., 2023; Radford et al., 2021) governing the sharpness of the induced distribution. Optimization uses stochastic gradient descent with momentum 0.9 (Dosovitskiy et al., 2021; Chen et al., 2020; He et al., 2015) and batch size 128 for 400,000 steps, executed on six NVIDIA L40S GPUs (48 GB each) under data parallelism (Paszke et al., 2019). The two hyperparameters that most directly shape the geometry—the temperature τ and the positive margin δ —were selected as described in Appendix A.2, which reports the complete training configuration.

Figure 1 visualizes the learned representation of the Base model through a two-dimensional UMAP projection (McInnes et al., 2020). Each point is an encoded position from a held-out test set, colored by its Stockfish-evaluated win probability for White: positions in which White holds a decisive advantage appear in gold (*White has checkmate*) and those in which Black holds a decisive advantage in dark green (*Black has checkmate*). The projection exhibits a continuous progression from one extreme to the other, confirming that the training objective induces the evaluation-aligned ordering the planning procedure presupposes—outcome similarity maps to spatial proximity, and advantage varies monotonically along a dominant direction of the space. We stress that this structure is inherited from the win-probability labels, and read the projection as evidence for the geometry that action selection exploits rather than as an independent measure of decision quality, which the playing-strength experiments assess directly.

3.4 EMBEDDING-GUIDED SEARCH

At inference, SOLIS plans directly in the learned space, ranking legal continuations along a single global advantage direction so that neither a rollout nor a learned value head is ever invoked. We first estimate the two extremes of the evaluative axis by averaging embeddings over held-out positions at the endpoints of the win-probability scale,

$$\mu_{\text{White}} = \mathbb{E}[z \mid p=1.0], \quad \mu_{\text{Black}} = \mathbb{E}[z \mid p=0.0],$$

and define the normalized advantage vector

$$\vec{a} = \frac{\mu_{\text{White}} - \mu_{\text{Black}}}{\|\mu_{\text{White}} - \mu_{\text{Black}}\|}.$$

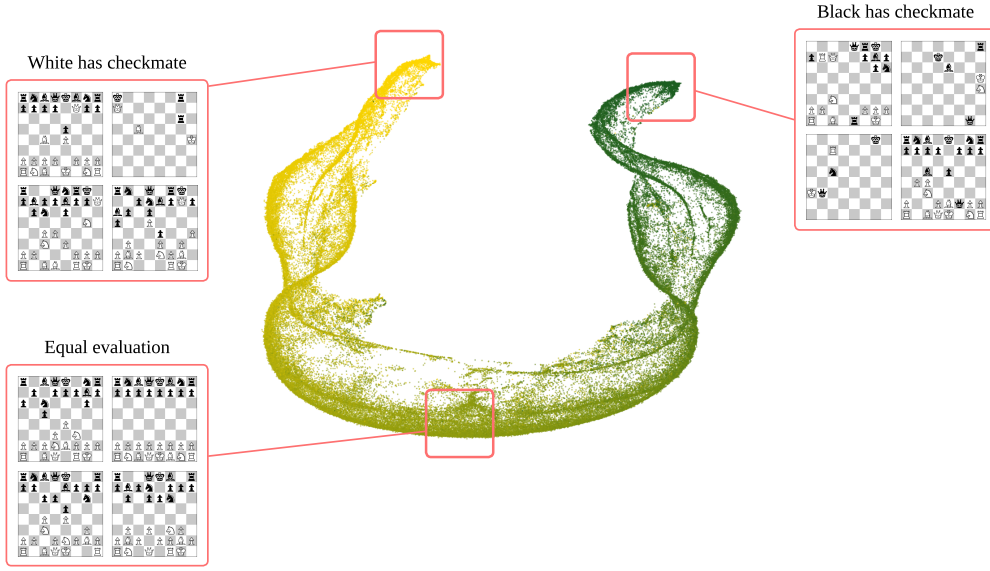


Figure 1: UMAP projection of the learned embedding space of the SOLIS Base model, with each held-out position colored by its Stockfish win probability (gold = White favored, green = Black favored). The smooth, unbroken gold-to-green gradient shows that the supervised contrastive objective organizes the space along a single coherent evaluative axis rather than into disjoint clusters—the continuity on which directional planning depends.

This unit vector $\vec{a}$ points from the losing region toward the winning region and serves as the sole planning signal. We convert it into scalar move scores through two complementary mechanisms that differ only in their choice of origin.

3.4.1 UNANCHORED (GLOBAL-DIRECTIONAL)

The first mechanism scores each child embedding by its raw alignment with the advantage direction. A child embedding z' receives the score

$$\text{score}_{\text{dir}}(z') = \langle z', \vec{a} \rangle,$$

favoring continuations whose embeddings project farthest along $\vec{a}$, independently of any reference origin. This is the mechanism depicted for illustration in Figure 2.

3.4.2 ANCHORED (RELATIVE PROJECTION)

The second mechanism translates each embedding by the Black mean before projecting, yielding

$$\text{score}_{\text{anch}}(z') = \langle z' - \mu_{\text{Black}}, \vec{a} \rangle,$$

so that the score equals the signed distance from μ_{Black} along $\vec{a}$. By fixing the origin at a semantically meaningful reference rather than at zero, anchoring eliminates dependence on an arbitrary coordinate origin and, as the experiments of Section 4 confirm, furnishes a more stable and consistently stronger basis for comparing continuations.

The search itself (Figure 2) is identical for both scoring mechanisms. Beginning from the “Starting board state”, we “Extract possible moves” and embed all children in parallel with the “Transformer encoder”; we then evaluate either $\text{score}_{\text{dir}}$ or $\text{score}_{\text{anch}}$ in the “Cosine similarity” block and retain the “Top- W ” children. Once the recursion reaches “ S levels deep?” it terminates and we “Select first move...”; otherwise the “Recursive search” continues under a min-max convention in which White maximizes and Black minimizes the advantage score. A transposition table backed by Zobrist hashing (Zobrist, 1990) caches previously encoded positions so that repeated states are never re-embedded. The procedure thus expands at most W children per node to depth S , bounding the total

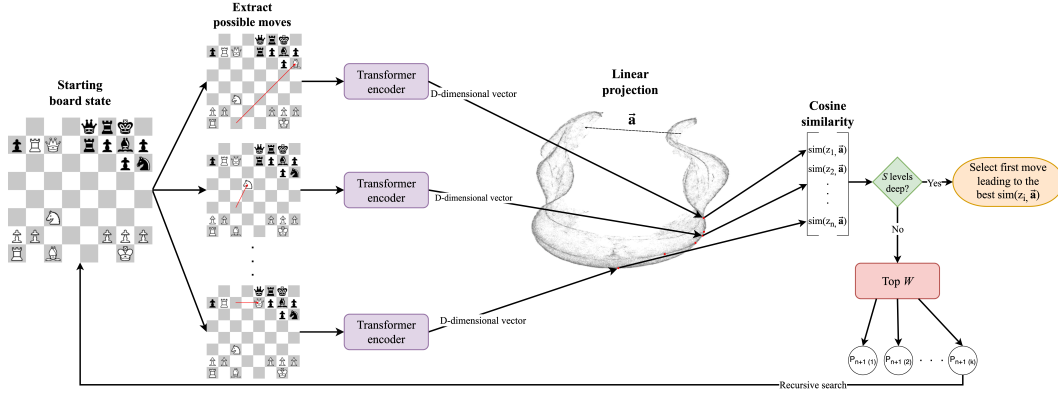


Figure 2: System overview of SOLIS. Candidate moves are extracted, embedded in parallel by the transformer encoder, and ranked by their alignment with the global advantage vector; the top- W children are expanded recursively to depth S under a min-max convention, and a Zobrist-hashed transposition table prevents any position from being re-embedded. For clarity the diagram illustrates the unanchored scoring mechanism $\cos(z', \vec{a})$; the anchored mechanism is identical apart from its origin.

expansion at $\sum_{i=0}^S W^i$ nodes—the compact and deterministic search budget whose consequences we quantify in Section 4.

4 RESULTS

4.1 RATING CALCULATION

We estimate Elo ratings by playing each SOLIS configuration against Stockfish under a fixed per-move time limit of 50 ms (Ruoss et al., 2024), with the complete Stockfish configuration reported in Appendix A.3. To disentangle the contributions of search breadth and depth, we conduct a systematic sweep over both dimensions: at every depth-width setting each model plays more than 600 games with alternating colors against Stockfish instances configured at a range of Elo caps, and both planning mechanisms—unanchored and anchored—are evaluated under identical conditions. The caps are chosen so that each configuration yields at least one matchup with a positive and one with a negative score, ensuring that the ratings are well identified. Elo ratings and their 95% confidence intervals are then estimated by Bayesian logistic regression with the BayesElo (Coulom, 2008) program at its default confidence parameter of 0.5. Figure 3 reports the estimates for the unanchored mechanism and Figure 4 those for the anchored mechanism; the Stockfish rating caps and the full match records underlying every estimate are tabulated in Appendix A.6 to permit exact reproduction. Because the opponent is time-limited Stockfish, these ratings measure strength relative to that baseline rather than an absolute skill level, and we interpret them accordingly.

Unanchored ratings Under the unanchored mechanism, Elo scales consistently and predictably with the search budget. Depth is the dominant driver of strength, whereas width produces a pronounced jump from $W \in \{1, 2\}$ to $W=3$ and negligible further gain at $W=5$. This early saturation is itself informative: because broader expansion recovers little additional strength, the strongest continuation almost always lies among the top three candidates, and SOLIS’s playing strength is therefore attributable to the quality of the learned geometry rather than to the breadth of the search. At shallow depths SOLIS Base leads Mini by roughly 100–250 Elo, but at depths 4 and 5 the far smaller Mini closes this gap and even overtakes Base at $W \in \{3, 5\}$ —direct evidence that, once sufficient depth is available, a compact 0.8M-parameter encoder matches a model $50\times$ its size.

Anchored ratings Anchored scoring produces smoother scaling in both depth and width, with the Base–Mini gap holding steady in the 100–250 Elo range. Both models surpass 2500 Elo at depth 5, and, in contrast to the unanchored case, Base remains consistently stronger than Mini across every setting—a monotonicity that indicates a fixed reference origin makes the larger model’s richer

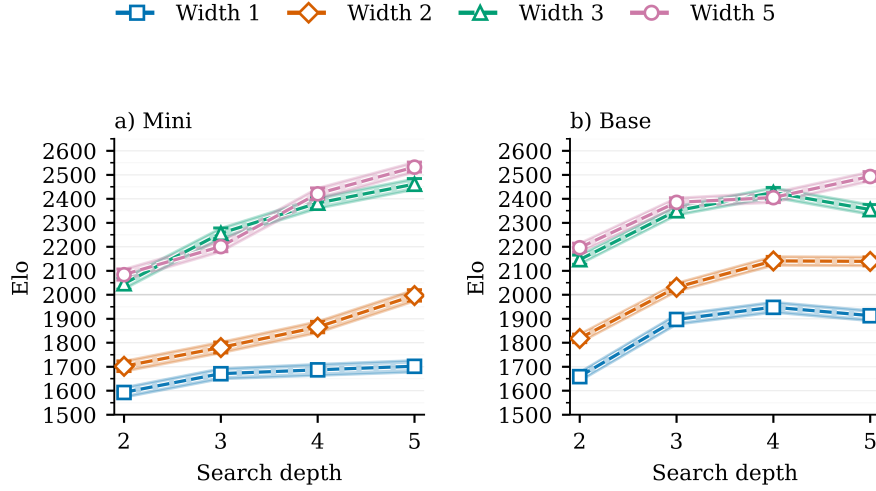


Figure 3: Estimated Elo of the SOLIS Mini and Base models across search widths W and depths S under the *unanchored* planning mechanism, obtained from more than 600 games per configuration against time-capped Stockfish; shaded bands denote 95% confidence intervals. Ratings scale consistently with the search budget, with depth contributing the largest gains and width saturating beyond $W=3$ —evidence that the strongest continuation almost always lies among the top three candidates.

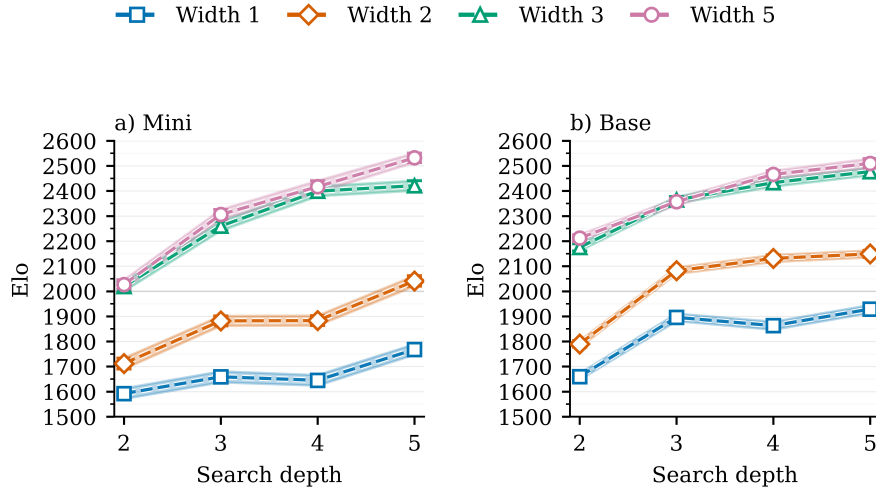


Figure 4: Estimated Elo of the SOLIS Mini and Base models across search widths W and depths S under the *anchored* planning mechanism, obtained from more than 600 games per configuration against time-capped Stockfish; shaded bands denote 95% confidence intervals. Ratings rise smoothly with both depth and width, and Base is uniformly stronger than Mini across all settings, so that the larger model's richer geometry becomes reliably exploitable once the origin is fixed at a meaningful reference.

geometry reliably exploitable, rather than intermittently so. Anchoring further yields a steady 10–30 Elo improvement for Base, confirming that projecting relative to the Black mean rather than the coordinate origin furnishes a more dependable evaluative basis.

4.2 EQUAL-DEPTH SIMULATIONS

To compare the two engines on an equal footing, we match their search depths directly. Guided by the scaling analysis of Section 4.1, we fix SOLIS’s branching factor at $W=3$, since wider searches yield only marginal gains and the strongest continuation reliably appears among the top three candidates. Stockfish, by contrast, is left to its default variable-width expansion with no imposed branching factor. This asymmetry is deliberate and works against our method: at equal depth Stockfish is granted unrestricted breadth while SOLIS is confined to a fixed learned pruning of three candidates, so any parity we observe is obtained under a handicap. At each depth $S \in \{3, 4, 5\}$ we play 200 games with alternating colors and report the outcomes for both scoring mechanisms in Table 2. The comparison isolates the effect of depth-matched search width; it does not control for per-node cost, an axis we discuss separately in Section 5.

Simulation results The equal-depth matches show that SOLIS is competitive with, and frequently superior to, an unrestricted-width Stockfish at the same depth. Under unanchored scoring SOLIS Base accumulates more match points than Stockfish at all three depths, while Mini draws level at depths 3 and 5 and trails only at depth 4; under anchored scoring Base loses only narrowly at depth 3 before overtaking Stockfish at depths 4 and 5, and Mini again wins at depths 3 and 5 while dipping below at depth 4. Across both mechanisms, then, SOLIS Base is favored in five of its six matchups, and the sole systematic exception—Mini at depth 4—is confined to the smallest encoder. That this parity holds despite the width handicap imposed on SOLIS makes the finding the more pointed: a learned three-way pruning of the move list is, at these depths, no weaker than Stockfish’s full-width expansion.

Table 2: Depth-matched match points for SOLIS versus Stockfish over 200 games per cell, with SOLIS fixed at width $W=3$ and Stockfish allowed its default unrestricted expansion—a comparison that handicaps SOLIS in search breadth. The middle block reports the unanchored mechanism (direct similarity with the global advantage vector) and the rightmost block the anchored mechanism (relative projection); higher is better and draws count as 0.5, so a score above 100 marks a match won on points. Bold entries mark the winner of each matchup: SOLIS Base leads at every depth under unanchored scoring and at depths 4 and 5 under anchored scoring, trailing only narrowly (98 to 102) at depth 3.

Config	Depth	Direct Similarity		Anchored Projection	
		SOLIS score	Stockfish score	SOLIS score	Stockfish score
Base	3	106	94	98	102
	4	109	91	108	92
	5	107.5	92.5	110	90
Mini	3	108	92	105	95
	4	85	115	91	109
	5	101	99	113	87

Nodes searched The efficiency of embedding-guided search is seen most directly in the two engines’ expansion budgets at equal depth. With a fixed branching factor $W=3$, SOLIS expands at most $N(S) = \sum_{i=0}^S 3^i$ nodes at depth S , giving $N(3) = 40$, $N(4) = 121$, and $N(5) = 364$. Stockfish, whose expansion is governed by pruning and search heuristics, explores a far larger and highly variable number: on average 137 nodes at depth 3 (maximum 1625), 214 at depth 4 (maximum 4463), and 338 at depth 5 (maximum 10611). SOLIS’s fixed budget therefore sits below Stockfish’s average at the shallower depths and one to two orders of magnitude below its worst-case expansion throughout, while retaining the parity in match points documented above. This node-count comparison substantiates the central empirical finding of the paper: **within the depths and branching factors we study, much of the strength conventionally obtained from broad tree expansion can instead be recovered from the quality of the learned representation**, so that a high-fidelity evaluation-aligned embedding substitutes for extensive search along the axis of node expansion. We report this efficiency in expanded nodes, the quantity the comparison controls; because a transformer encoder evaluation and a handcrafted Stockfish node differ in per-node cost, the

result speaks to search economy rather than to wall-clock or hardware efficiency, a distinction we take up in Section 5.

4.3 INTERPRETABLE GAME TRAJECTORIES

Beyond raw playing strength, the learned space affords a degree of interpretability that rollout- and value-based planners do not readily provide, since the very quantity guiding the search—displacement along the advantage axis—is itself visualizable. To make this concrete, we trace the latent trajectories of complete games through the embedding, shown in Figure 5 for three representative outcomes: a game won by White, a balanced draw, and a game won by Black. Each game’s positions are embedded independently, projected onto the same two-dimensional map, and linked by arrows marking their temporal progression. The trajectories exhibit a clear and consistent structure: decisive games sweep smoothly and near-monotonically toward the winning region, whereas closely contested games oscillate about the neutral center. These paths complement the static embedding of Figure 1 by rendering the planning process itself visible—they show how successive move selections translate into displacement toward the region corresponding to the eventual outcome—so that the same geometry that drives the search also explains it. We offer these trajectories as illustrative case studies of three outcomes rather than as an aggregate measure of decision quality, which the playing-strength results quantify.

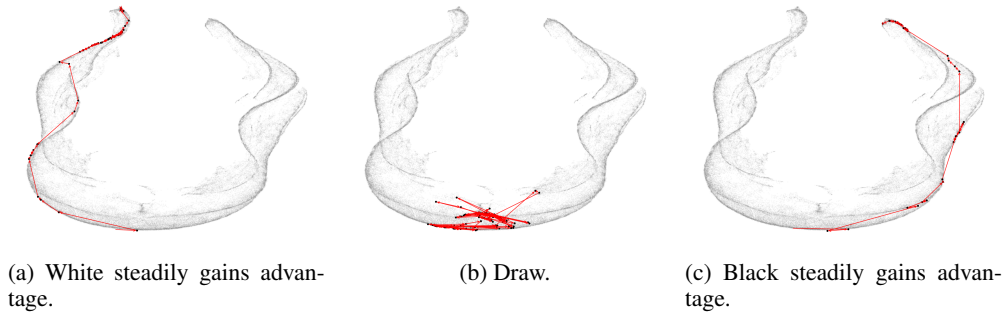


Figure 5: Latent trajectories of three representative games embedded in the shared representation space, with red arrows marking the temporal progression of positions. Decisive games (a, c) advance smoothly toward the corresponding winning region, while the drawn game (b) remains near the neutral center, illustrating that the planning dynamics are legible directly from the geometry. These are hand-selected case studies chosen to span the three outcome types, not a statistical summary.

5 DISCUSSION

5.1 LIMITATIONS

We frame the boundaries of the present study explicitly, both to delimit the claims above and to organize the natural extensions that follow from them. Four considerations bear directly on how the results should be read.

Supervision by, and evaluation against, Stockfish SOLIS learns its geometry from Stockfish win-probability labels and is benchmarked chiefly against Stockfish configurations, so the experiments establish that a contrastively learned representation can substitute for broad tree expansion at matched depth rather than that SOLIS captures chess structure independent of its supervising evaluator. Our node-count and equal-depth results are properly read as statements about search economy relative to this baseline; disentangling genuine planning ability from the distillation of a static evaluation would require baselines and opponents we did not run, and we return to both below.

Scope of the comparison and absent baselines The comparisons reported here place SOLIS against time-limited and depth-matched Stockfish; they do not include a supervised scalar value head, a direct regressor to the Stockfish evaluation, or a policy/value network trained on the same

5M labeled positions and used to rank child positions. Consequently our evidence speaks to the sufficiency of an evaluation-aligned contrastive geometry for competitive shallow-search play, not to its superiority over standard supervised value learning; whether the contrastive objective and vector arithmetic are essential, or whether the same global advantage direction would emerge from a simpler regression or classification target, is an open question that a controlled ablation—contrastive versus regression or classification, with variation in the number and definition of positives, the margin δ , the temperature τ , and the reference anchor, and comparison against random or PCA directions—is the natural means to settle.

Domain and evaluation density We develop the method entirely within chess, a perfect-information domain in which legal moves, transitions, and dense evaluation labels are externally available. Our results substantiate evaluation-aligned latent planning at the shallow depths and small branching factors studied here; they do not by themselves establish that the approach replaces rollout- or tree-search-based planning in general, nor that it transfers to MDPs in which evaluation labels are sparse, delayed, or unreliable. We regard these as the load-bearing conditions of the method and treat transfer beyond them as a hypothesis for future work rather than a demonstrated capability.

Node efficiency versus wall-clock cost Our efficiency claims are expressed in expanded nodes, the quantity the equal-depth comparison controls. Because a transformer encoder evaluation is considerably more expensive per node than a handcrafted Stockfish evaluation, node-count parity need not translate into wall-clock or energy parity on given hardware; a like-for-like timing comparison, which we did not conduct, would be required to characterize that dimension.

Blindness to threefold repetition SOLIS selects moves purely by alignment with the advantage vector, which is precisely what makes its decision-making effective at minimal overhead; the same design choice, however, discards move history, with a concrete failure mode as its consequence: blindness to threefold repetition. Because a position recurring three times—with identical side to move, castling rights, and en passant rights—permits either player to claim a draw, an astute opponent can deliberately engineer repetitions, for instance through perpetual checks, to neutralize a winning position that SOLIS would otherwise convert. Game C in Appendix A.4 illustrates exactly this: SOLIS reaches a winning position against Stockfish yet concedes a premature draw by repetition. This limitation is a direct and well-understood consequence of the non-Markovian tokenization, evidence that the current state representation is not Markovian in the rule-complete sense; the corresponding remedy—augmenting the state with rule-aware history features (repetition and fifty-move counters, castling rights, and en passant state) together with repetition-aware move selection—is conceptually straightforward, and we note its measurement as an open item below.

Indecisiveness in winning positions A second limitation arises because the evaluation target does not encode the length of a forcing sequence: a mate-in-one and a mate-in-twenty both carry a win probability of 1.0, even though the shorter line is plainly preferable. When many continuations share the maximal score, SOLIS may select a longer one, leaving the opponent room to complicate the position and occasionally forfeiting a deserved win to a draw or even a loss. Ruoss et al. (2024) met the same difficulty and circumvented it by deferring to Stockfish as an external oracle; SOLIS instead commits to its own representation, preserving the method’s self-contained character at the cost of this residual gap. Game I in Appendix A.5 exhibits the phenomenon, with SOLIS holding numerous winning continuations over several consecutive moves yet drifting into a draw. Explicitly preferring shorter forced wins is a natural extension that would close this gap directly.

5.2 CONCLUSION

We set out to determine whether planning can be conducted directly in an evaluation-aligned latent space, without training a policy, a value head, or a dynamics model, and within the chess setting we study our results answer this question affirmatively. We introduced SOLIS, which learns a supervised contrastive embedding in which proximity reflects similarity of outcome and a single global *advantage vector* points from losing toward winning regions, so that planning reduces to vector arithmetic: candidate actions are ranked by their alignment with this direction under a shallow search.

Instantiated in chess with a fixed branching factor $W=3$ and shallow depths $S \in \{3, 4, 5\}$, SOLIS matches or exceeds a time-capped Stockfish in depth-matched play while expanding far fewer nodes per move—40 to 364 for SOLIS against Stockfish averages in the hundreds and worst cases in the thousands at the same depths, and this despite Stockfish’s unrestricted search width. Elo scales cleanly with depth; under anchored scoring both Mini and Base surpass 2500 Elo at depth five, with anchoring contributing a consistent 10–30 Elo gain for Base. That width confers little benefit beyond three candidates is itself a substantive finding, for it localizes the source of SOLIS’s strength in the quality of the learned space rather than in the breadth of expansion, and the parity of the 0.8M-parameter Mini with a model $50\times$ its size at sufficient depth reinforces the same conclusion. The embedding additionally yields readable game paths, with decisive games advancing steadily toward the winning region and draws lingering near neutral.

Taken together, and read against the conditions that produced them—dense, externally available evaluation labels, a perfect-information domain, and node-counted rather than wall-clock efficiency—these results position evaluation-aligned latent planning as a simple, interpretable, and node-efficient alternative to rollout-based search and explicit dynamics models in this regime. Because the approach requires only that states admit an available or learnable evaluation, it is a natural candidate for other perfect-information games and for control tasks with large action spaces in which a low-branching search over a well-structured representation is advantageous; establishing that transfer, however, is a task for future work rather than a claim of the present study. The most immediate directions follow from the boundaries drawn in Section 5: ablating the contrastive objective against regression and classification baselines and against a supervised value head trained on the same labels; isolating the strength contributed by the encoder alone through one-ply child selection without recursion; incorporating rule-aware state features and repetition-aware move selection; preferring shorter forced wins; reporting wall-clock and hardware-controlled efficiency; combining SOLIS with policy priors or MCTS; and testing transfer to non-Stockfish opponents, human game positions, and domains beyond chess.

REPRODUCIBILITY STATEMENT

We document the study in sufficient detail to reproduce its procedures. Section 3 specifies the encoder architecture, input representation, supervised contrastive objective, and inference procedure, with model configurations summarized in Table 1; the complete training setup, including all hyperparameters and the temperature and margin selection, appears in Appendix A.2. The evaluation protocol—match setup, the exact Stockfish configuration, and the rating calculation—is detailed in Section 4 and Appendix A.3, and the full win–draw–loss records underpinning every Elo estimate are tabulated in Appendix A.6. The training data are the publicly available ChessBench positions and the opponent is the official Stockfish 16 release, both introduced above. Beyond these specifications, releasing the training code, trained checkpoints, data-preprocessing scripts, and match-running scripts would further ease exact replication, and we regard doing so as part of the intended artifact for this work.

REFERENCES

- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations, 2020. URL <https://arxiv.org/abs/2002.05709>.
- S. Chowdhury, G. Hamerly, and M. McGarrity. Active learning strategy using contrastive learning and k-means for aquatic invasive species recognition. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 848–858, 2024. doi: 10.1109/WACVW60836.2024.00097.
- R. Coulom. Whole-history rating: A bayesian rating system for players of time-varying strength. In *Computers and Games*, volume 5131 of *LNCS*, pages 113–124. Springer, 2008. doi: 10.1007/978-3-540-87608-3_10.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.

- A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL <https://arxiv.org/abs/2010.11929>.
- R. Gieselmann and F. T. Pokorny. Latent planning via expansive tree search. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=zSdz5scsnzU>.
- R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra. Imagebind: One embedding space to bind them all, 2023. URL <https://arxiv.org/abs/2305.05665>.
- D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels, 2019. URL <https://arxiv.org/abs/1811.04551>.
- D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination, 2020. URL <https://arxiv.org/abs/1912.01603>.
- D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models, 2022. URL <https://arxiv.org/abs/2010.02193>.
- D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models, 2024. URL <https://arxiv.org/abs/2301.04104>.
- J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, T. Pfaff, T. Weber, L. Buesing, and P. W. Battaglia. Combining q-learning and search with amortized value estimates, 2020. URL <https://arxiv.org/abs/1912.02807>.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus), 2023. URL <https://arxiv.org/abs/1606.08415>.
- M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization, 2021. URL <https://arxiv.org/abs/1906.08253>.
- Z. Jiang, T. Zhang, M. Janner, Y. Li, T. Rocktäschel, E. Grefenstette, and Y. Tian. Efficient planning in a compact latent action space, 2023. URL <https://arxiv.org/abs/2208.10291>.
- P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan. Supervised contrastive learning, 2021. URL <https://arxiv.org/abs/2004.11362>.
- M. Lai. Giraffe: Using deep reinforcement learning to play chess, 2015. URL <https://arxiv.org/abs/1509.01549>.
- L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020. URL <https://arxiv.org/abs/1802.03426>.
- T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker. Model-based reinforcement learning: A survey, 2022. URL <https://arxiv.org/abs/2006.16712>.
- Y. Nasu. Efficiently updatable neural-network-based evaluation functions for computer shogi, 2018. Accessed: 2024-04-25.
- G.-C. Pascutto and G. Linscott. Leelachesszero, 2018. URL <https://lczero.org>. Accessed: 2024-04-25.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.

-
- M. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 2014. ISBN 9781118625873. URL <https://books.google.com/books?id=VvBjBAAQBAJ>.
- A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision, 2021. URL <https://arxiv.org/abs/2103.00020>.
- T. Romstad, M. Costalba, J. Kiiski, G. Linscott, Y. Nasu, M. Isozaki, and H. Noda. Stockfish, 2008. URL <https://stockfishchess.org>. Accessed: 2025-04-22.
- A. Ruoss, G. Delétang, S. Medapati, J. Grau-Moya, L. K. Wenliang, E. Catt, J. Reid, C. A. Lewis, J. Veness, and T. Genewein. Amortized planning with large-scale transformers: A case study on chess, 2024. URL <https://arxiv.org/abs/2402.04494>.
- S. Russell and P. Norvig. *Artificial Intelligence, A modern approach*. 1995.
- J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, Dec. 2020. ISSN 1476-4687. doi: 10.1038/s41586-020-03051-4. URL <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- C. E. Shannon. XXII. programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, Jan. 2014. ISSN 1532-4435.
- R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- J. v. Neumann. Zur theorie der gesellschaftsspiele. *Mathematische annalen*, 100(1):295–320, 1928.
- A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding, 2019. URL <https://arxiv.org/abs/1807.03748>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- A. L. Zobrist. A new hashing method with application for game playing. *ICGA Journal*, 13(2): 69–73, 1990.

A APPENDIX

A.1 USE OF LARGE LANGUAGE MODELS (LLMs)

Large language models were employed solely for minor auxiliary tasks such as formatting and figure or table placement. They were not used to design experiments, analyze results, or generate technical content; all methodological and experimental contributions were developed and verified exclusively by the authors.

A.2 TRAINING SETUP

Both SOLIS configurations (Mini and Base) are trained with the supervised contrastive objective on 5M ChessBench positions, using cosine similarity over ℓ_2 -normalized embeddings and SGD with momentum on six 48 GB L40S GPUs for 400k steps at batch size 128.

The temperature τ and the positive margin δ are the two hyperparameters that most directly shape the induced geometry. The temperature $\tau = 0.07$ follows established contrastive practice (Radford et al., 2021; Girdhar et al., 2023), in which smaller values sharpen the similarity distribution while larger ones oversmooth the gradients; sweeping $\tau \in \{0.05, 0.07, 0.1\}$, we found that 0.07 converged most reliably across both model sizes. The margin $\delta = 0.05$ balances two competing failure modes—a large δ collapses disparate positions into a single neighborhood, whereas a small δ fragments the space—while additionally absorbing Stockfish evaluation noise, since win-probability differences below 0.05 are rarely meaningful in practice. A systematic sweep over δ and over the number of positives per anchor, which we fix at five, would further characterize the sensitivity of the learned geometry to these choices; we report the settings used and leave that sensitivity analysis to future work.

Table 3: Complete SOLIS training hyperparameters, shared across the Mini and Base configurations.

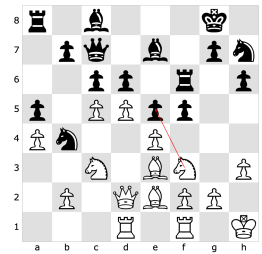
Attribute	Value
Loss	SupCon
τ	0.07
δ (positive margin)	0.05
Positives per anchor	5
Optimizer	SGD
Momentum	0.9
Learning rate	0.05
Batch size	128
Steps	400k
Dropout	0.10
Activation	GELU
Similarity	cosine
Embedding norm	ℓ_2
Token length	77

A.3 STOCKFISH SETUP

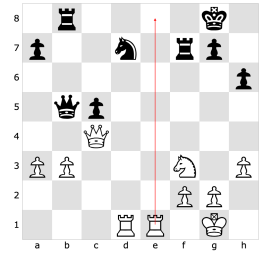
Across all experiments we use the official Linux release of Stockfish 16 (Romstad et al., 2008) with a 32 MB hash and default threading. Rating caps are imposed through the `UCI_LimitStrength` and `UCI_Elo` settings, and fixed search depths through the `depth` parameter, so that the fixed-time and equal-depth regimes of Section 4 differ only in this control. To measure the nodes expanded for a given move we set the engine information level to `INFO_ALL` and read the reported `nodes` attribute, which is the quantity underlying the node-count comparison of Section 4.

A.4 REPRESENTATIVE GAMES: STOCKFISH (CAP) VS. SOLIS (DEPTH, WIDTH)

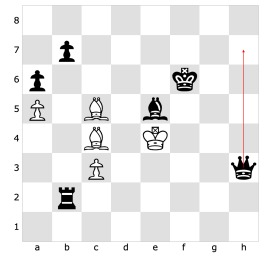
We present sample games between SOLIS and Stockfish in which SOLIS is limited to search depth S and width W while Stockfish searches without a fixed branching factor under a rating cap. These are illustrative case studies rather than a representative sample: Games *A*, *B*, and *E* display SOLIS's tactical acuity and long-term planning, whereas Games *C* and *D* instantiate the failure modes analyzed in Section 5. In each table the left column records the game in Portable Game Notation (PGN) and the right column depicts a critical moment.

<pre> 1. d4 Nf6 2. c4 d6 3. Nc3 e5 4. Nf3 Qe7 5. e4 c6 6. d5 Qc7 7. Be2 h6 8. a4 a5 9. O-O Be7 10. h3 Na6 11. Be3 Nb4 12. Kh1 Nh7 13. Qd2 O-O 14. Rad1 f5 15. c5 Rf6 16. Nxe5 f4 17. Bxf4 Rxf4 18. Qxf4 dxe5 19. d6 exf4 20. Bc4+ Kf8 21. dxc7 Ke8 22. e5 Ng5 23. Rd6 Bxd6 24. exd6 Bd7 25. Re1+ Kf8 26. Re7 Bf5 27. h4 Nd5 28. Nxd5 cxd5 29. Bxd5 Be6 30. Bxe6 f3 31. hxg5 b6 32. Bh3 hxg5 33. Re5 Rc8 34. Rf5+ Kg8 35. Rf7 g4 36. cxb6 g6 37. d7 fxg2+ 38. Kh2 Rf8 39. Bxg2 Kxf7 40. Bc6 Ke7 41. Kg3 Rf3+ 42. Kh2 Rd3 43. b7 Rh3+ 44. Kg2 Re3 45. d8=Q+ Kf7 46. b8=Q Rg3+ 47. Kh2 Rh3+ 48. Kg1 Rh7 49. Qe8+ Kf6 50. Qbd8+ Kf5 51. Be4+ Kf4 52. Qf6# 1-0 </pre>	
--	---

Game A — SOLIS ($S=5$, $W=3$) vs. SF 2350, SOLIS–White. Result: 1–0. SOLIS sacrifices a knight in the middlegame and converts the resulting initiative into a decisive advantage.

<pre> 1. d4 d5 2. e3 c6 3. Bd3 h6 4. h3 Nf6 5. Nf3 e6 6. O-O b6 7. c4 Ba6 8. b3 Nbd7 9. Bb2 Bb4 10. a3 Be7 11. Nbd2 O-O 12. Re1 c5 13. Qe2 Qc7 14. e4 Nxe4 15. Nxe4 dxe4 16. Bxe4 Rab8 17. d5 Bf6 18. dxe6 Bxb2 19. exf7+ Rxf7 20. Qxb2 Bb7 21. Bxb7 Qxb7 22. Rad1 b5 23. Qe2 bxc4 24. Qxc4 Qb5 25. Re8+ Nf8 26. Rxb8 Qxb8 27. Rd5 Qf4 28. Qxf4 Rxf4 29. Rxc5 Ne6 30. Rc6 Nd4 31. Nxd4 a5 32. Rc4 Rf8 33. Nc6 Kh8 34. Nxa5 Ra8 35. b4 Rd8 36. g3 Rd3 37. a4 g5 38. b5 Rd2 39. b6 Rd5 40. b7 Rxa5 41. Rc8+ Kh7 42. b8=Q Rd5 43. Rc7+ Kg6 44. Qg8+ Kf5 45. Qxd5+ Kg6 46. Qf7# 1-0 </pre>	
--	--

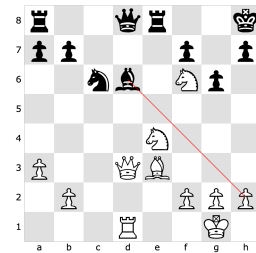
Game B — SOLIS ($S=5$, $W=3$) vs. SF 2300, SOLIS–White. Result: 1–0. SOLIS sacrifices a rook to win Stockfish's queen and converts the material advantage into a win.

<pre> 1. e3 Nf6 2. Nc3 d5 3. Nf3 c5 4. d4 e6 5. g3 a6 6. a4 Nc6 7. Ne2 Bd6 8. Bg2 O-O 9. h3 Re8 10. c3 Qc7 11. Nd2 e5 12. Nb3 c4 13. dxe5 Nxe5 14. Nbd4 Nd3+ 15. Kf1 Nc5 16. Bf3 Bd7 17. Kg2 Be5 18. a5 Rad8 19. Qc2 g6 20. Bd2 h5 21. Be1 h4 22. gxh4 Kg7 23. Ng3 Rh8 24. h5 Rdg8 25. hxg6 fxg6 26. Be2 Qd6 27. Qd1 Qe7 28. f3 Bd6 29. Nf1 Rh5 30. h4 Rgh8 31. Bf2 R5h7 32. Qe1 Bb8 33. Bg3 Ba7 34. Bf2 Nh5 35. Ng3 Bb8 36. f4 Nf6 37. h5 Rh6 38. hxg6 Rxh1 39. Qxh1 Qe8 40. Ndf5+ Bxf5 41. Nxf5+ Kxg6 42. Ne7+ Kf7 43. e4 Rxh1 44. Rxh1 Qxe7 45. Bxc5 Qxe4+ 46. Bf3 Qg6+ 47. Kf1 Bxf4 48. b4 cxb3 49. Rh8 Ke6 50. Rf8 b2 51. Rxf6+ Kxf6 52. Bxd5 b1=R+ 53. Ke2 Rb2+ 54. Kf3 Be5 55. Bc4 Qh5+ 56. Ke3 Qh3+ 57. Ke4 Qh7+ 58. Kf3 Qh3+ 59. Ke4 Qh7+ 60. Kf3 Qh3+ 61. Ke4 Qh7+ 62. Kf3 Qh3+ 63. Ke4 Qh7+ 64. Kf3 Qh3+ 65. Ke4 1/2-1/2 </pre>	
--	---

Game C — SF 2200 vs. SOLIS ($S=5$, $W=3$), SOLIS–Black. Result: 1/2–1/2. SOLIS reaches a winning endgame but concedes a draw, illustrating the threefold-repetition blindness analyzed in Section 5.

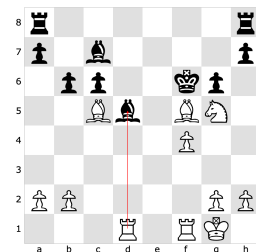
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971

1. d4 d5 2. c4 dxc4 3. e3 e5 4. Bxc4 exd4 5. exd4
Nf6 6. Nc3 Bd6 7. Qe2+ Qe7 8. Be3 O-O 9. Nf3 c6 10.
O-O Be6 11. Rfd1 Re8 12. d5 cxd5 13. Bxd5 Nc6 14.
a3 Nxd5 15. Nxd5 Qd8 16. Qd3 Bg4 17. Ng5 g6 18. Ne4
Bxd1 19. Ndf6+ Kh8 20. Rxd1 Bxh2+ 21. Kxh2 Qxd3 22.
Rxd3 Red8 23. Nd7 Kg7 24. f3 h6 25. Rd6 f5 26. Nec5
Re8 27. Bd2 Re2 28. Kh1 Kf7 29. b3 Ke7 30. Nxb7 Rc8
31. Ndc5 Kf7 32. Kg1 Kg8 33. Rxg6+ Kf7 34. Rd6 Ne7
35. Rd7 Rg8 36. Nd6+ Kg6 37. Nc4 Kh5 38. Ne3 f4 39.
Ne6 fxe3 40. Nf4+ Kg5 41. Nxe2 exd2 42. Kf2 Kf6 43.
Nc3 Ke6 44. Rd4 Rc8 45. Ne4 d1=N+ 46. Rxd1 Rc2+ 47.
Kg1 Nf5 48. Rd2 Rxd2 49. Nxd2 Ke5 50. Kh2 Kf4 51.
b4 Ke3 52. Ne4 Nd4 53. a4 Nc2 54. b5 Nd4 55. Nd6
Ne6 56. a5 Kd3 57. g4 Kd4 58. b6 axb6 59. axb6 Nc5
60. Kg3 Kd5 61. b7 Na6 62. Nf7 Kc6 63. Nxb6 Kxb7
64. f4 Kc7 65. Kh4 Kd7 66. Kh5 Nc5 67. f5 Kd6 68.
f6 Ne6 69. Nf7+ Kc6 70. Kh6 Kd5 71. g5 Nc5 72. Kg6
Ne6 73. Kh5 Kd4 74. g6 Kc4 75. Ne5+ Kd5 76. Ng4 Ke4
77. Kh6 Kf5 78. g7 Nxg7 79. Kxg7 Kxg4 80. Kf7 Kf5
81. Ke7 Kg5 82. f7 Kf4 83. Kf6 Kf3 84. Ke5 Kg3 85.
Ke4 Kf2 86. Kd3 Kg1 87. Ke3 Kh2 88. f8=Q Kg3 89.
Qg8+ Kh3 90. Kf4 Kh2 91. Kf3 Kh3 92. Qg3# 1-0



Game D — SF 2500 vs. SOLIS ($S=5$, $W=3$), SOLIS-Black. Result: 1-0 SOLIS commits a short-sighted piece sacrifice that leaves it with a lasting material deficit.

1. d4 d5 2. e3 Nd7 3. c4 e5 4. dxe5 dxc4 5. Nf3
Qe7 6. Bxc4 Nxe5 7. Nxe5 Qxe5 8. Qb3 Qh5 9. e4
Nf6 10. O-O Bd6 11. Qb5+ c6 12. Qxh5 Nxh5 13. Nc3
Be5 14. f4 Bc7 15. Be3 g6 16. Bb3 Be6 17. Rad1 Ng7
18. Bc2 f5 19. Bc5 Kf7 20. exf5 Nxf5 21. Ne4 b6
22. Ng5+ Kf6 23. Bxf5 Bd5 24. Rxd5 cxd5 25. Bd4+
Kxf5 26. h3 Bxf4 27. Bxh8 Kxg5 28. Bg7 Rg8 29. Bd4
Re8 30. Rf3 Re1+ 31. Kf2 Re6 32. g3 Bd2 33. Rf7
Re4 34. Bf6+ Kh6 35. g4 a6 36. Kg3 d4 37. h4 Be1+
38. Kh3 Re3+ 39. Kg2 Bxh4 40. Bxh4 a5 41. Bf6 a4
42. Rd7 Rd3 43. Rd5 b5 44. Rxb5 a3 45. Re5 Rd2+
46. Kf3 axb2 47. g5+ Kh5 48. Rb5 Rd3+ 49. Kf4 h6
50. gxh6+ Kxh6 51. Rxb2 g5+ 52. Kg4 Rd1 53. Bxg5+
Kg6 54. Rb6+ Kg7 55. Kf5 Kf7 56. Rb7+ Ke8 57. a4
Ra1 58. Re7+ Kf8 59. Rh7 Rxa4 60. Bh6+ Kg8 61. Re7
Rc4 62. Kg5 Rc5+ 63. Kg4 Ra5 64. Rg7+ Kh8 65. Re7
d3 66. Rd7 d2 67. Bxd2 Rc5 68. Bf4 Kg8 69. Be3 Rb5
70. Bh6 Re5 71. Kf4 Rc5 72. Ke4 Rb5 73. Kf4 Rb4+
74. Ke5 Rb1 75. Kf6 Rb2 76. Kg6 Rb6+ 77. Kh5 Rb2
78. Bg5 Rb6 79. Bh6 Re6 80. Kg5 Ra6 81. Rg7+ Kh8
82. Re7 Ra5+ 83. Kg6 Ra6+ 84. Kh5 Kg8 85. Rb7 Rc6
86. Rd7 Rc5+ 87. Bg5 Rc2 88. Kg6 Rc6+ 89. Bf6 Re6
90. Rd8+ Re8 91. Rxe8# 1-0

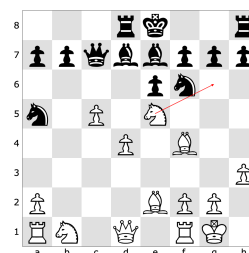


Game E — SOLIS ($S=5$, $W=3$) vs. Stockfish 2500 , SOLIS-White. Result: 1-0 SOLIS gives up the exchange for long-term positional compensation and converts the resulting advantage into a win.

A.5 EQUAL-DEPTH MATCHES: SOLIS D vs. STOCKFISH D

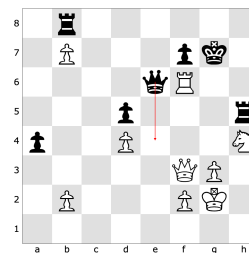
We include several illustrative games in which SOLIS and Stockfish search to the same depth D , to show that SOLIS's latent planning competes head-to-head with an uncapped-width Stockfish on equal-depth terms. As before, the left column of each table records the game in Portable Game Notation (PGN) and the right column depicts a critical moment; these games are selected for illustration and are not an aggregate performance measure.

1. d4 d5 2. e3 c5 3. Nf3 Nf6 4. Be2 e6 5. O-O Nc6 6. b3 cxd4 7. exd4 Be7 8. c3 Qc7 9. h3 Bd7 10. c4 dxc4 11. bxc4 Na5 12. Ne5 Rd8 13. Bf4 Bd6 14. c5 Be7 15. Ng6 Qc8 16. Nxh8 Bc6 17. Nc3 Kf8 18. Rc1 Qa8 19. Bc7 b5 20. Bxa5 Bxg2 21. d5 Bxh3 22. Qd3 Bf5 23. Qg3 b4 24. Rfd1 Bxc5 25. Qc7 Be7 26. Nb5 Nxd5 27. Rxd5 Qxd5 28. Bxb4 Qd7 29. Qxd7 Re8 30. Rc8 g5 31. Qxe7+ Kg7 32. Qxf7+ Kh6 33. Bf8+ Rxf8 34. Qxf8# 1-0



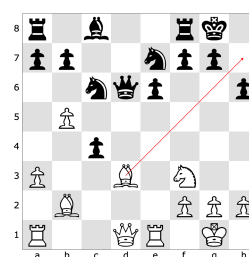
Game F — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 SOLIS sacrifices a knight in the middlegame for a concrete tactical advantage and goes on to win.

1. d4 d5 2. e3 c5 3. Nf3 e6 4. Be2 cxd4 5. exd4 Bd6 6. O-O Nf6 7. Bg5 Nbd7 8. Nbd2 h6 9. Bh4 O-O 10. Re1 a5 11. Bf1 Bf4 12. c3 Qb6 13. Qc2 Qc7 14. a4 b6 15. Bb5 Ba6 16. Bxa6 Rxa6 17. Qd3 Ra7 18. Re2 Qc8 19. Reel Nh5 20. Nf1 Nh6 21. N1d2 Nh5 22. Nf1 Bc7 23. Qd1 Nh6 24. Qd3 Nh5 25. Qd1 Nh6 26. Qd3 Rb7 27. Qb5 Ra7 28. N1d2 Nh5 29. h3 g5 30. Rac1 gxf4 31. c4 Nf4 32. Rc3 Qd8 33. g3 Nxf3+ 34. Kg2 Ng5 35. cxd5 h3+ 36. Kg1 h2+ 37. Kxh2 exd5 38. Rc6 Nf6 39. Nxf5 hxg5 40. Kg2 Re8 41. Rh1 Bd6 42. Nf3 Kg7 43. Nxf5 Rc7 44. Rxd6 Qxd6 45. Qf1 b5 46. axb5 Rcc8 47. Rh4 Rh8 48. Nf3 Ne4 49. Rf4 Rh5 50. Qe2 Qe6 51. Nh4 Rch8 52. b6 Nf6 53. Qf3 a4 54. b7 Rb8 55. Rxf6 Qe4 56. Rxf7+ Kh8 57. Qxe4 dxe4 58. Rc7 Rg5 59. Rc8+ Rg8 60. Rc7 Rg5 61. Rc8+ Rg8 62. Ng6+ Kg7 63. Ne7 Rf8 64. Rc7 Rf7 65. d5 Rbf8 66. Nf5+ Kg6 67. Rxf7 Kxf7 68. Nd6+ Kg6 69. Nc8 Kg5 70. b8=Q Kg4 71. Qc7 Rf5 72. Nd6 e3 73. Qg7+ Rg5 74. f3+ Kh5 75. Qh7# 1-0



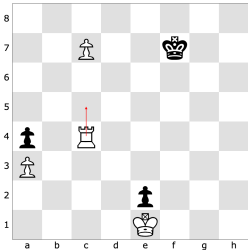
Game G — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 Stockfish steadily builds an advantage but blunders in the late middlegame; SOLIS capitalizes on the error and prevails in the endgame.

1. d4 c6 2. e4 d5 3. Nc3 Nf6 4. e5 Ng8 5. Bd3 e6 6. Nf3 c5 7. dxc5 Bxc5 8. O-O Nc6 9. a3 h6 10. b4 Bb6 11. Nb5 Bc7 12. Re1 Bb8 13. c4 Nge7 14. Nd6+ Bxd6 15. exd6 Qxd6 16. Bb2 O-O 17. b5 dxc4 18. Bh7+ Kh8 19. Qxd6 f6 20. Be4 c3 21. Bxc3 e5 22. bxc6 Bf5 23. Qxe7 Bxe4 24. Rxe4 bxc6 25. Nxe5 Rfe8 26. Nf7+ Kg8 27. Nxh6+ Kh7 28. Rae1 Rad8 29. Ba5 Rd4 30. g4 Rxe4 31. Rxe4 Rb8 32. Qxa7 Rd8 33. Rf4 Rd3 34. g5 fxg5 35. Rd4 Rxa3 36. Kf1 Rb3 37. Rb4 Rc3 38. h4 gxh4 39. Rd4 h3 40. Qe7 h2 41. Qe4+ Kxh6 42. Rd6+ Kg5 43. Rg6+ Kh5 44. Qg4# 1-0



Game H — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS-White. Result: 1-0 SOLIS sacrifices a bishop to launch a discovered attack on Stockfish's queen and converts the advantage into a win.

1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079

1. d4 d5 2. e3 c6 3. c4 e6 4. Nf3 Nf6 5. Be2 c5 6. O-O Nc6 7. b3 a6 8. Bb2 Be7 9. a3 cxd4 10. exd4 O-O 11. c5 b6 12. b4 bxc5 13. dxc5 Bb7 14. Nbd2 a5 15. b5 Bxc5 16. Rc1 Qb6 17. bxc6 Ng4 18. Rxc5 Ba6 19. Rc2 Nxf2 20. Rxf2 Bxe2 21. Qxe2 a4 22. c7 Rac8 23. Bd4 Qb5 24. Qxb5 Rfe8 25. Qb8 h6 26. Ne5 g5 27. h4 Rf8 28. Ne4 dxe4 29. Rxf7 Rxb8 30. Rxf8+ Rxf8 31. hxc6 Rc8 32. gxh6 e3 33. Bxe3 Kh7 34. Rc4 Kg8 35. Kh2 Kh7 36. g4 Rg8 37. g5 Ra8 38. Rg4 Kh8 39. Ba7 Rf8 40. Bb8 Rf2+ 41. Kg1 Rc2 42. Rc4 Rb2 43. Ba7 Rb1+ 44. Kg2 Kh7 45. Bf2 Rb2 46. g6+ Kxh6 47. Kh2 Rxf2+ 48. Kg3 Rg2+ 49. Kxg2 Kg7 50. Rc1 Kf6 51. Kh2 Kg7 52. Nd7 e5 53. Nf6 Kxf6 54. Rg1 Kg7 55. Rf1 Kh6 56. Rg1 Kg7 57. Rf1 Kxg6 58. Rc1 Kg7 59. Rc4 Kf7 60. Kg2 Ke7 61. Kf2 Kd6 62. Ke1 Kd5 63. Rc1 Ke4 64. Ke2 Kd4 65. Rc2 e4 66. Rc1 Kd5 67. Ke1 Ke5 68. Rc4 Kd5 69. Rc2 Kd6 70. Rc1 Ke7 71. Rc4 Kf7 72. Rc5 e3 73. Kf1 Ke7 74. Rc4 Kf7 75. Kg2 e2 76. Kf2 Ke7 77. Ke1 Kf7 78. Rc5 Kg7 79. Rc2 Kf6 80. Rc4 Kf7 81. Rc5 Kg6 82. Kd2 Kf7 83. Ke1 Ke7 84. Rc4 Kf7 85. Rc5 Kg6 86. Rc4 Kf7 87. Rc5 1/2-1/2	
--	--

Game I — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS-White. Result: 1/2-1/2 SOLIS struggles to plan when every continuation is winning, exemplifying the indecisiveness in won positions discussed in Section 5.

A.6 SOLIS VS. STOCKFISH RESULTS

To support reproducibility of our Elo estimates, we report the complete match outcomes between SOLIS and Stockfish, with SOLIS searching at depths $S \in \{2, 3, 4, 5\}$. Each table is indexed by Stockfish’s Elo cap (rows) and SOLIS’s search width W (columns), and each entry gives the win–draw–loss counts of the matchup with the total match points in parentheses. Separate tables are provided for the Base and Mini models under both the unanchored and anchored scoring mechanisms, so that every point in Figures 3 and 4 can be traced to its underlying games.

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32–46–22 (55)	41–47–12 (64)	–	–	1500	–	–	–	–
1550	39–38–23 (58)	37–50–13 (62)	–	–	1550	–	–	–	–
1600	32–44–24 (54)	33–51–16 (58)	–	–	1600	–	–	–	–
1650	33–41–26 (54)	29–46–25 (52)	–	–	1650	–	–	–	–
1700	34–38–28 (53)	22–53–25 (48)	–	–	1700	–	–	–	–
1750	37–35–28 (54)	30–49–21 (54)	–	–	1750	–	–	–	–
1800	29–42–29 (50)	36–42–22 (57)	–	–	1800	39–29–32 (54)	37–49–14 (62)	–	–
1850	–	–	–	–	1850	35–32–33 (51)	30–50–20 (55)	–	–
1900	–	–	–	–	1900	37–30–33 (52)	40–41–19 (60)	–	–
1950	–	–	–	–	1950	31–28–41 (45)	37–44–19 (59)	–	–
2000	–	–	34–40–26 (54)	43–31–26 (58)	2000	41–27–32 (54)	33–45–22 (56)	–	–
2050	–	–	46–38–16 (65)	36–38–26 (55)	2050	39–27–34 (52)	31–41–28 (52)	–	–
2100	–	–	34–45–21 (56)	34–31–35 (50)	2100	41–27–32 (54)	36–38–26 (55)	–	–
2150	–	–	46–31–23 (62)	30–43–27 (52)	2150	–	–	–	–
2200	–	–	31–35–34 (48)	35–28–37 (49)	2200	–	–	33–42–25 (54)	30–41–29 (50)
2250	–	–	34–34–32 (51)	40–29–31 (54)	2250	–	–	35–41–24 (56)	33–43–24 (54)
2300	–	–	32–32–36 (48)	37–34–29 (54)	2300	–	–	35–33–32 (52)	36–34–30 (53)
2350	–	–	–	–	2350	–	–	37–37–26 (56)	29–37–34 (48)
2400	–	–	–	–	2400	–	–	34–36–30 (52)	37–38–25 (56)
2450	–	–	–	–	2450	–	–	37–28–35 (51)	43–29–28 (58)
2500	–	–	–	–	2500	–	–	41–30–29 (56)	34–30–36 (49)
2550	–	–	–	–	2550	–	–	–	–
2600	–	–	–	–	2600	–	–	–	–
2650	–	–	–	–	2650	–	–	–	–
2700	–	–	–	–	2700	–	–	–	–

Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	–	–	–	–	1600	–	–	–	–
1650	–	–	–	–	1650	–	–	–	–
1700	–	–	–	–	1700	–	–	–	–
1750	–	–	–	–	1750	–	–	–	–
1800	–	–	–	–	1800	–	–	–	–
1850	–	–	–	–	1850	–	–	–	–
1900	–	–	–	–	1900	–	–	–	–
1950	37–26–37 (50)	41–42–17 (62)	–	–	1950	36–34–30 (53)	38–41–21 (58)	–	–
2000	43–20–37 (53)	31–43–26 (52)	–	–	2000	31–28–41 (45)	27–52–21 (53)	–	–
2050	38–27–35 (52)	40–38–22 (59)	–	–	2050	41–24–35 (53)	36–37–27 (54)	–	–
2100	42–25–33 (54)	32–48–20 (56)	–	–	2100	43–23–34 (54)	36–39–25 (56)	–	–
2150	36–23–41 (48)	31–47–22 (54)	–	–	2150	42–23–35 (54)	27–45–28 (50)	–	–
2200	46–13–41 (52)	39–39–22 (58)	–	–	2200	42–21–37 (52)	23–51–26 (48)	–	–
2250	–	–	–	–	2250	–	–	–	–
2300	–	–	–	–	2300	–	–	–	–
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2450	–	–	40–39–21 (60)	39–38–23 (58)	2450	–	–	25–46–29 (48)	43–30–27 (58)
2500	–	–	38–38–24 (57)	41–40–19 (61)	2500	–	–	40–36–24 (58)	34–44–22 (56)
2550	–	–	44–20–36 (54)	42–29–29 (56)	2550	–	–	40–31–29 (56)	38–38–24 (57)
2600	–	–	46–17–37 (54)	37–23–40 (48)	2600	–	–	46–25–29 (58)	33–37–30 (52)
2650	–	–	41–18–41 (50)	44–22–34 (55)	2650	–	–	47–17–36 (56)	30–35–35 (48)
2700	–	–	45–16–39 (53)	42–22–36 (53)	2700	–	–	38–25–37 (50)	45–21–34 (56)

Figure 6: Full match records for the SOLIS Base model against Stockfish under *unanchored* planning, arranged as per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Entries report win–draw–loss counts with total match points in parentheses, out of 100 games per cell.

1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	32–32–36 (48)	27–45–28 (50)	–	–	1350	–	–	–	–
1400	30–41–29 (50)	33–27–40 (46)	–	–	1400	–	–	–	–
1450	29–44–27 (51)	33–40–27 (53)	–	–	1450	–	–	–	–
1500	31–38–31 (50)	31–39–30 (50)	–	–	1500	29–38–33 (48)	28–43–29 (50)	–	–
1550	38–35–27 (56)	23–50–27 (48)	–	–	1550	41–36–23 (59)	36–37–27 (54)	–	–
1600	34–30–36 (49)	29–37–34 (48)	–	–	1600	37–30–33 (52)	37–30–33 (52)	–	–
1650	24–45–31 (46)	28–36–36 (46)	–	–	1650	32–38–30 (51)	30–40–30 (50)	–	–
1700	–	–	–	–	1700	28–37–35 (46)	21–43–36 (42)	–	–
1750	–	–	–	–	1750	36–33–31 (52)	28–40–32 (48)	–	–
1800	–	–	37–31–32 (52)	37–36–27 (55)	1800	39–24–37 (51)	29–35–36 (46)	–	–
1850	–	–	34–24–42 (46)	34–31–35 (50)	1850	–	–	–	–
1900	–	–	40–28–32 (54)	40–29–31 (54)	1900	–	–	–	–
1950	–	–	39–29–32 (54)	39–29–32 (54)	1950	–	–	–	–
2000	–	–	38–34–28 (55)	42–28–30 (56)	2000	–	–	34–35–31 (52)	38–21–41 (48)
2050	–	–	41–27–32 (54)	40–26–34 (53)	2050	–	–	38–34–28 (55)	34–33–33 (50)
2100	–	–	35–30–35 (50)	36–26–38 (49)	2100	–	–	33–34–33 (50)	34–29–37 (48)
2150	–	–	–	–	2150	–	–	31–35–34 (48)	38–25–37 (50)
2200	–	–	–	–	2200	–	–	38–18–44 (47)	31–26–43 (44)
2250	–	–	–	–	2250	–	–	36–23–41 (48)	45–26–29 (58)
2300	–	–	–	–	2300	–	–	40–22–38 (51)	34–26–40 (47)
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2500	–	–	–	–	2500	–	–	–	–
Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	–	–	–	–	1350	–	–	–	–
1400	–	–	–	–	1400	–	–	–	–
1450	–	–	–	–	1450	–	–	–	–
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	36–42–22 (57)	35–31–34 (50)	–	–	1600	–	–	–	–
1650	29–40–31 (49)	36–33–31 (52)	–	–	1650	–	–	–	–
1700	33–34–33 (50)	36–32–32 (52)	–	–	1700	39–30–31 (54)	34–34–32 (51)	–	–
1750	43–26–31 (56)	28–38–34 (47)	–	–	1750	37–29–34 (52)	32–30–38 (47)	–	–
1800	38–24–38 (50)	30–37–33 (48)	–	–	1800	42–20–38 (52)	33–31–36 (48)	–	–
1850	49–23–28 (60)	29–33–38 (46)	–	–	1850	36–24–40 (48)	43–28–29 (57)	–	–
1900	–	–	–	–	1900	45–26–29 (58)	46–31–23 (62)	–	–
1950	–	–	–	–	1950	40–22–38 (51)	37–35–28 (54)	–	–
2000	–	–	–	–	2000	–	–	–	–
2050	–	–	–	–	2050	–	–	–	–
2100	–	–	–	–	2100	–	–	–	–
2150	–	–	31–31–38 (46)	44–21–35 (54)	2150	–	–	–	–
2200	–	–	35–29–36 (50)	48–27–25 (62)	2200	–	–	27–37–36 (46)	33–38–29 (52)
2250	–	–	37–31–32 (52)	37–28–35 (51)	2250	–	–	30–33–37 (46)	38–31–31 (54)
2300	–	–	27–37–36 (46)	36–34–30 (53)	2300	–	–	38–37–25 (56)	33–39–28 (52)
2350	–	–	22–42–36 (43)	37–35–28 (54)	2350	–	–	32–29–39 (46)	29–36–35 (47)
2400	–	–	36–29–35 (50)	37–26–37 (50)	2400	–	–	30–26–44 (43)	30–32–38 (46)
2500	–	–	–	–	2500	–	–	26–34–40 (43)	43–25–32 (56)

Figure 7: Full match records for the SOLIS Mini model against Stockfish under *unanchored* planing, arranged as per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Entries report win–draw–loss counts with total match points in parentheses, out of 100 games per cell.

1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32–46–22 (55)	41–47–12 (64)	–	–	1500	–	–	–	–
1550	39–38–23 (58)	37–50–13 (62)	–	–	1550	–	–	–	–
1600	32–44–24 (54)	33–51–16 (58)	–	–	1600	–	–	–	–
1650	33–41–26 (54)	29–46–25 (52)	–	–	1650	–	–	–	–
1700	34–38–28 (53)	22–53–25 (48)	–	–	1700	–	–	–	–
1750	37–35–28 (54)	30–49–21 (54)	–	–	1750	–	–	–	–
1800	29–42–29 (50)	36–42–22 (57)	–	–	1800	39–29–32 (54)	37–49–14 (62)	–	–
1850	–	–	–	–	1850	35–32–33 (51)	30–50–20 (55)	–	–
1900	–	–	–	–	1900	37–30–33 (52)	40–41–19 (60)	–	–
1950	–	–	–	–	1950	31–28–41 (45)	37–44–19 (59)	–	–
2000	–	–	34–40–26 (54)	43–31–26 (58)	2000	41–27–32 (54)	33–45–22 (56)	–	–
2050	–	–	46–38–16 (65)	36–38–26 (55)	2050	39–27–34 (52)	31–41–28 (52)	–	–
2100	–	–	34–45–21 (56)	34–31–35 (50)	2100	41–27–32 (54)	36–38–26 (55)	–	–
2150	–	–	46–31–23 (62)	30–43–27 (52)	2150	–	–	–	–
2200	–	–	31–35–34 (48)	35–28–37 (49)	2200	–	–	33–42–25 (54)	30–41–29 (50)
2250	–	–	34–34–32 (51)	40–29–31 (54)	2250	–	–	35–41–24 (56)	33–43–24 (54)
2300	–	–	32–32–36 (48)	37–34–29 (54)	2300	–	–	35–33–32 (52)	36–34–30 (53)
2350	–	–	–	–	2350	–	–	37–37–26 (56)	29–37–34 (48)
2400	–	–	–	–	2400	–	–	34–36–30 (52)	37–38–25 (56)
2450	–	–	–	–	2450	–	–	37–28–35 (51)	43–29–28 (58)
2500	–	–	–	–	2500	–	–	41–30–29 (56)	34–30–36 (49)
2550	–	–	–	–	2550	–	–	–	–
2600	–	–	–	–	2600	–	–	–	–
2650	–	–	–	–	2650	–	–	–	–
2700	–	–	–	–	2700	–	–	–	–
Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	–	–	–	–	1600	–	–	–	–
1650	–	–	–	–	1650	–	–	–	–
1700	–	–	–	–	1700	–	–	–	–
1750	–	–	–	–	1750	–	–	–	–
1800	–	–	–	–	1800	–	–	–	–
1850	–	–	–	–	1850	–	–	–	–
1900	–	–	–	–	1900	–	–	–	–
1950	37–26–37 (50)	41–42–17 (62)	–	–	1950	36–34–30 (53)	38–41–21 (58)	–	–
2000	43–20–37 (53)	31–43–26 (52)	–	–	2000	31–28–41 (45)	27–52–21 (53)	–	–
2050	38–27–35 (52)	40–38–22 (59)	–	–	2050	41–24–35 (53)	36–37–27 (54)	–	–
2100	42–25–33 (54)	32–48–20 (56)	–	–	2100	43–23–34 (54)	36–39–25 (56)	–	–
2150	36–23–41 (48)	31–47–22 (54)	–	–	2150	42–23–35 (54)	27–45–28 (50)	–	–
2200	46–13–41 (52)	39–39–22 (58)	–	–	2200	42–21–37 (52)	23–51–26 (48)	–	–
2250	–	–	–	–	2250	–	–	–	–
2300	–	–	–	–	2300	–	–	–	–
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2450	–	–	40–39–21 (60)	39–38–23 (58)	2450	–	–	25–46–29 (48)	43–30–27 (58)
2500	–	–	38–38–24 (57)	41–40–19 (61)	2500	–	–	40–36–24 (58)	34–44–22 (56)
2550	–	–	44–20–36 (54)	42–29–29 (56)	2550	–	–	40–31–29 (56)	38–38–24 (57)
2600	–	–	46–17–37 (54)	37–23–40 (48)	2600	–	–	46–25–29 (58)	33–37–30 (52)
2650	–	–	41–18–41 (50)	44–22–34 (55)	2650	–	–	47–17–36 (56)	30–35–35 (48)
2700	–	–	45–16–39 (53)	42–22–36 (53)	2700	–	–	38–25–37 (50)	45–21–34 (56)

Figure 8: Full match records for the SOLIS Base model against Stockfish under *anchored* planning, arranged as per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Entries report win–draw–loss counts with total match points in parentheses, out of 100 games per cell.

1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	36–32–32 (52)	39–30–31 (54)	–	–	1350	–	–	–	–
1400	23–49–28 (48)	30–32–38 (46)	–	–	1400	–	–	–	–
1450	33–37–30 (52)	24–47–29 (48)	–	–	1450	–	–	–	–
1500	34–38–28 (53)	25–40–35 (45)	–	–	1500	29–40–31 (49)	35–33–32 (52)	–	–
1550	28–38–34 (47)	40–39–21 (60)	–	–	1550	34–35–31 (52)	37–29–34 (52)	–	–
1600	23–47–30 (46)	28–42–30 (49)	–	–	1600	32–36–32 (50)	32–39–29 (52)	–	–
1650	35–29–36 (50)	38–37–25 (56)	–	–	1650	34–30–36 (49)	37–33–30 (54)	–	–
1700	–	–	–	–	1700	36–30–34 (51)	34–35–31 (52)	–	–
1750	–	–	–	–	1750	34–25–41 (46)	34–30–36 (49)	–	–
1800	–	–	35–37–28 (54)	40–28–32 (54)	1800	32–29–39 (46)	32–31–37 (48)	–	–
1850	–	–	32–32–36 (48)	44–29–27 (58)	1850	–	–	–	–
1900	–	–	37–30–33 (52)	45–21–34 (56)	1900	–	–	–	–
1950	–	–	43–30–27 (58)	36–29–35 (50)	1950	–	–	–	–
2000	–	–	35–31–34 (50)	43–27–30 (56)	2000	–	–	46–32–22 (62)	38–27–35 (52)
2050	–	–	37–29–34 (52)	45–27–28 (58)	2050	–	–	37–27–36 (50)	39–26–35 (52)
2100	–	–	36–27–37 (50)	42–34–24 (59)	2100	–	–	29–34–37 (46)	48–17–35 (56)
2150	–	–	–	–	2150	–	–	37–33–30 (54)	39–25–36 (52)
2200	–	–	–	–	2200	–	–	37–31–32 (52)	41–23–36 (52)
2250	–	–	–	–	2250	–	–	36–23–41 (48)	44–20–36 (54)
2300	–	–	–	–	2300	–	–	30–43–27 (52)	38–26–36 (51)
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2500	–	–	–	–	2500	–	–	–	–
Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	–	–	–	–	1350	–	–	–	–
1400	–	–	–	–	1400	–	–	–	–
1450	–	–	–	–	1450	–	–	–	–
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	26–45–29 (48)	31–33–36 (48)	–	–	1600	–	–	–	–
1650	37–39–24 (56)	28–32–40 (44)	–	–	1650	–	–	–	–
1700	23–39–38 (42)	29–41–30 (50)	–	–	1700	37–36–27 (55)	34–29–37 (48)	–	–
1750	36–29–35 (50)	38–30–32 (53)	–	–	1750	42–23–35 (54)	37–33–30 (54)	–	–
1800	35–24–41 (47)	33–36–31 (51)	–	–	1800	36–28–36 (50)	31–35–34 (48)	–	–
1850	41–21–38 (52)	30–28–42 (44)	–	–	1850	38–21–41 (48)	33–32–35 (49)	–	–
1900	–	–	–	–	1900	37–27–36 (50)	37–37–26 (56)	–	–
1950	–	–	–	–	1950	43–18–39 (52)	38–24–38 (50)	–	–
2000	–	–	–	–	2000	–	–	–	–
2050	–	–	–	–	2050	–	–	–	–
2100	–	–	–	–	2100	–	–	–	–
2150	–	–	39–23–38 (50)	32–30–38 (47)	2150	–	–	–	–
2200	–	–	33–40–27 (53)	38–25–37 (50)	2200	–	–	30–40–30 (50)	32–34–34 (49)
2250	–	–	36–31–33 (52)	30–27–43 (44)	2250	–	–	29–43–28 (50)	49–26–25 (62)
2300	–	–	32–33–35 (48)	37–28–35 (51)	2300	–	–	31–30–39 (46)	30–35–35 (48)
2350	–	–	35–34–31 (52)	40–25–35 (52)	2350	–	–	26–32–42 (42)	40–34–26 (57)
2400	–	–	28–26–46 (41)	26–38–36 (45)	2400	–	–	39–32–29 (55)	31–32–37 (47)
2500	–	–	–	–	2500	–	–	30–29–41 (44)	28–41–31 (48)

Figure 9: Full match records for the SOLIS Mini model against Stockfish under *anchored* planning, arranged as per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Entries report win–draw–loss counts with total match points in parentheses, out of 100 games per cell.